

Functions

Day – 14

Start by explaining that while LLMs are great at conversation, their knowledge is frozen in time and they can't perform real-world actions. Function calling is the key technique that allows us to connect an LLM to external tools and data, giving it "superpowers" to interact with the world.

What is Function Calling?

Function calling is a technique where you give a Large Language Model (LLM) a list of tools (functions) it can use. When a user asks a question, the model doesn't just answer with text; it can instead reply with a structured JSON object saying, "I need to run this specific function with these specific arguments to answer your question."

Your application's code then receives this JSON, executes the actual function (e.g., calls a weather API), and sends the result back to the LLM to formulate a natural language response.

Analogy: Think of an LLM as a brilliant but office-bound manager. Function calling gives this manager a phone and a list of employees (your functions) they can call to get things done in the outside world.

The Three Core Components:

Function calling involves a three-step conversation between your application and the model.

1. Describe Your Functions (The Tools List):

- First, you define your available tools in your code. For an LLM to use them, you must describe them in a specific format (usually JSON Schema).
- This description tells the model:
 - The function's name (e.g., `get_current_weather`).
 - A clear description of what it does (e.g., "Gets the current weather for a specific location").
 - The parameters it requires (e.g., `location` as a string) and which ones are mandatory.

2. The User's Prompt (The Request):

- The user asks a question as they normally would.
- You send this prompt to the model **along with the list of function descriptions**.
- **Example Prompt:** "What's the weather like in Ludhiana?"

3. The Model's Response (The Plan):

- The model analyzes the prompt and sees that it can't answer from its internal knowledge.
- It checks the list of functions you provided and realizes the `get_current_weather` function can help.
- Instead of a text answer, it sends back a structured JSON object telling your application which function to call and with what arguments.

Example: Functions.py

```
function_prompt = """
```

You are a helpful assistant. You have access to the following tools:

1. `get_user_name()` - returns the user's name
2. `get_weather(city)` - returns weather for a city
3. `get_today_date()` - returns today's date
4. `get_current_time()` - returns the current time

If the question requires multiple answers, call all relevant functions.

Respond ONLY with lines like:

CALL: `get_weather(city="Delhi")`

5. `get_joke()` - tells a joke

6. `get_news()` - gives today's top news

7. `get_cricket_news()` - shows latest cricket news headlines

CALL: `get_cricket_news()`

CALL: `get_current_time()`

Do NOT explain or say anything else. Only
time?

A:

CALL: `get_weather(city="Mumbai")` use `CALL:` per line.

Examples:

Q: What is the weather in Mumbai and current

CALL: get_current_time()

App.py:

```
from flask import Flask, request, jsonify, render_template
import os

from dotenv import load_dotenv

import google.generativeai as genai

from functions import *

from prompts import function_prompt

# Load environment variables

load_dotenv()

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

# Initialize Gemini model

model = genai.GenerativeModel("gemini-2.0-flash")

# Flask app

app = Flask(__name__)

# Mapping Gemini function names to real Python functions

function_map = {

    "get_user_name": get_user_name,

    "get_weather": get_weather,

    "get_today_date": get_today_date,

    "get_current_time": get_current_time,

    "get_joke": get_joke,

    "get_news": get_news,

    "get_cricket_news": get_cricket_news

}

# Homepage to load the query UI

@app.route("/", methods=["GET"])

def home():

    return render_template("index.html")
```

Route to handle user queries

```
@app.route("/ask", methods=["POST"])

def ask():

    user_query = request.json.get("query")

    full_prompt = f"{function_prompt}\nQ: {user_query}\nA:"

    try:

        response = model.generate_content(full_prompt)

        output = response.text.strip()

        if "CALL:" in output:

            lines = output.splitlines()

            results = []

            for line in lines:

                if line.startswith("CALL:"):

                    call_text = line.replace("CALL:", "").strip()

                    fn_name = call_text.split("(")[0]

                    args_part = call_text[call_text.find("(")+1 : call_text.rfind(")")]

                    args_dict = eval(f"dict({args_part})") if args_part else {}

                    # Call the actual function

                    try:

                        result = function_map[fn_name](**args_dict) if args_dict else
function_map[fn_name]()

                        results.append(f"✅ {fn_name}: {result}")

                    except Exception as e:

                        results.append(f"❌ Error in {fn_name}: {str(e)}")

            return jsonify({"response": "\n".join(results)})

        # If Gemini doesn't return CALLs, just show the plain response

        return jsonify({"response": output})

    except Exception as e:

        return jsonify({"error": str(e)})
```

Run the app

```
if __name__ == "__main__":  
    app.run(debug=True)
```

Prompt evaluation techniques are methods used to measure the quality, accuracy, and reliability of responses generated by a Large Language Model (LLM) for a given prompt. These techniques help determine which prompt structures, wording, and parameters produce the best results for a specific task.

1. Human Evaluation 🧑

Human evaluation is the gold standard for assessing prompt performance, especially for tasks involving creativity, nuance, or subjective quality. It involves people rating the model's outputs based on a set of criteria.

- **How it works:** You create a "golden dataset" of ideal inputs and outputs. Raters then score the model's responses against this standard using methods like A/B testing (comparing two outputs side-by-side) or Likert scales (e.g., rating relevance from 1 to 5).
- **Pros:** It's the most accurate way to measure real-world usefulness, context, and safety.
- **Cons:** It's slow, expensive, and can be inconsistent due to human subjectivity.

2. Model-Based Evaluation 🤖

Model-based evaluation uses another powerful LLM (often a "judge" model like GPT-4) to score the output of the model being tested. The judge model is given the original prompt, the generated response, and a set of instructions on how to score it.

- **How it works:** You prompt a judge model with a rubric to evaluate the target model's output. For example: "You are an expert evaluator. Rate the following response on a scale of 1-10 for accuracy. Here is the prompt: [...], and here is the response: [...]."
- **Pros:** It's much faster and cheaper than human evaluation and can be easily scaled.
- **Cons:** It can suffer from bias (e.g., a judge model may favor responses similar to its own style) and may not be as reliable for highly nuanced or specialized tasks.

3. Evaluation Metrics 📊

For tasks with objective right or wrong answers, you can use automated, mathematical metrics to score performance.

- **Common Metrics:**

- **Exact Match:** The model's output must exactly match the reference answer. Useful for simple Q&A.
 - **BLEU (Bilingual Evaluation Understudy):** Measures how many n-grams (phrases of 1 to 4 words) in the model's output overlap with a reference text. Often used for translation and text summarization.
 - **ROUGE (Recall-Oriented Understudy for Gisting Evaluation):** Similar to BLEU but focuses on recall, measuring how much of the reference text is captured in the output. Also used for summarization.
 - **Semantic Similarity:** Uses embeddings to measure if the *meaning* of the model's output is close to the reference answer, even if the exact words are different.
- **Pros:** These methods are fast, consistent, and easy to automate for large-scale testing.
 - **Cons:** They often fail to capture the semantic nuance or creative quality of a response. An answer can be factually correct but score poorly on BLEU/ROUGE if it uses different wording.